

Práctica 2: Desarrollo de backend con Python, arquitectura limpia e Inteligencia Artificial

Contexto de la práctica

El objetivo de esta segunda práctica es reemplazar el backend original por uno nuevo desarrollado íntegramente en Python (utilizando Flask o FastAPI). El frontend completo en Svelte 5 desarrollado en la práctica anterior deberá conectarse a este nuevo backend sin requerir modificaciones importantes en su lógica de negocio o consumo de API.

El backend tiene dos recursos principales:

- Productos (CRUD completo, protegido por roles).
- Usuarios (CRUD completo, con roles usuario/admin).

El foco principal de esta entrega es la correcta separación de responsabilidades en el servidor y el uso documentado y crítico de herramientas de Inteligencia Artificial como asistentes de desarrollo.

Requisitos mínimos de la entrega (5 puntos)

Estos requisitos son obligatorios para aprobar (máximo 5 puntos si solo se cumple este bloque).

1. Estructura y separación de responsabilidades

- Backend creado con Flask o FastAPI.
- Organización modular del código en capas claras. Por ejemplo: routers/controllers (manejo de peticiones HTTP), services (lógica de negocio) y repositories/models (acceso a datos).
- Prohibido tener toda la lógica centralizada en el archivo principal de enrutamiento.

2. Autenticación básica con JWT

- Generación y validación de tokens JWT en el nuevo backend.
- Reproducir la lógica necesaria para que el envío de credenciales al backend siga funcionando correctamente desde el frontend.
- Protección de rutas privadas mediante dependencias o middlewares, gestionando respuestas de error (ej. 401, 403).

3. Migración del API (contrato de interfaz)

- El backend debe exponer los mismos endpoints (URLs y métodos HTTP) que la versión anterior para el listado, creación, edición y borrado de recursos.
- La estructura JSON de entrada y salida debe ser compatible con la que ya espera la aplicación Svelte 5.

Uso de IA en el desarrollo (hasta 2 puntos)

Este bloque evalúa la capacidad del alumno para utilizar la IA (Gemini, ChatGPT, Copilot, etc.) de forma profesional y reflexiva, no como un mero generador de código ciego.

Registro de prompts e iteraciones (hasta 1 punto)

- Inclusión de un documento (Markdown) detallando los prompts clave utilizados para generar o refactorizar partes complejas del backend (ej. configuración de JWT, estructuración en capas).
- Explicación de cómo se refinó el prompt cuando el primer resultado no fue satisfactorio.

Análisis crítico (hasta 1 punto)

- Documentar al menos un "error" o "alucinación" que haya cometido la IA durante el desarrollo.
- Explicar detalladamente por qué el código generado era incorrecto o subóptimo (ej. fallos de seguridad, mala separación de responsabilidades) y cómo se corrigió manualmente aplicando los conceptos vistos en clase.

Funcionalidades avanzadas de Backend (hasta 3 puntos)

Cada funcionalidad suma parte de los puntos máximos según su grado de calidad y complejidad técnica.

Validación estricta de datos (hasta 1 punto)

- Uso de librerías nativas o del framework (como Pydantic en FastAPI o Marshmallow/WTForms en Flask) para validar los datos de entrada en la creación y edición de recursos.
- Retorno automático de mensajes de error estructurados (ej. 422 Unprocessable Entity) si los datos no cumplen los formatos o rangos exigidos.

Manejo global de excepciones (hasta 1 punto)

- Implementación de un manejador global de errores en el framework para capturar excepciones de lógica de negocio o base de datos y traducirlas a respuestas HTTP limpias y unificadas.

Persistencia en Base de Datos y patrón repositorio (hasta 1 punto)

- Sustituir las estructuras de datos en memoria por un motor de base de datos real.
- Recomendado: Utilizar SQLite mediante un ORM estándar (SQLAlchemy o SQLModel) para minimizar la configuración del entorno.
- Alternativa libre: Se permite el uso de otras bases de datos (PostgreSQL, MongoDB, etc.) siempre y cuando el acceso a datos esté estrictamente encapsulado en la capa de repositorios y no contamine la lógica de negocio ni los controladores.
- *Nota:* Queda prohibido el uso de persistencia simulada (archivos de texto plano, JSON o arrays en memoria).

Criterios de evaluación

Aspecto	Descripción	Puntos máx.
Requisitos mínimos del backend	API funcional, separación de responsabilidades en capas, autenticación JWT compatible.	5
Desarrollo con IA: registro	Documentación de prompts y evolución del desarrollo asistido.	1
Desarrollo con IA: análisis	Identificación de errores de la IA y justificación de las correcciones manuales.	1
Avanzado: validaciones y errores	Uso de esquemas de validación (Pydantic/Marshmallow) y manejo global de excepciones.	2
Avanzado: Base de Datos (ORM)	Persistencia real y uso de patrón repositorio.	1
Total		10

Entrega

- Código en un repositorio público (GitHub, GitLab, etc.) con instrucciones claras de instalación y ejecución en el README.
- Documento anexo con la memoria del uso de Inteligencia Artificial.
- Indicar qué partes del backend se están utilizando (endpoints principales y roles necesarios).
- Necesario subir la documentación al Campus.
- Fecha de entrega: Hasta el 2 de junio de 2026 a las 23:59.